



DFS 길찾기 알고리즘

<https://youtu.be/TjvubAFAWtA?feature=shared>

<https://youtu.be/c-F07IQHURE?feature=shared>

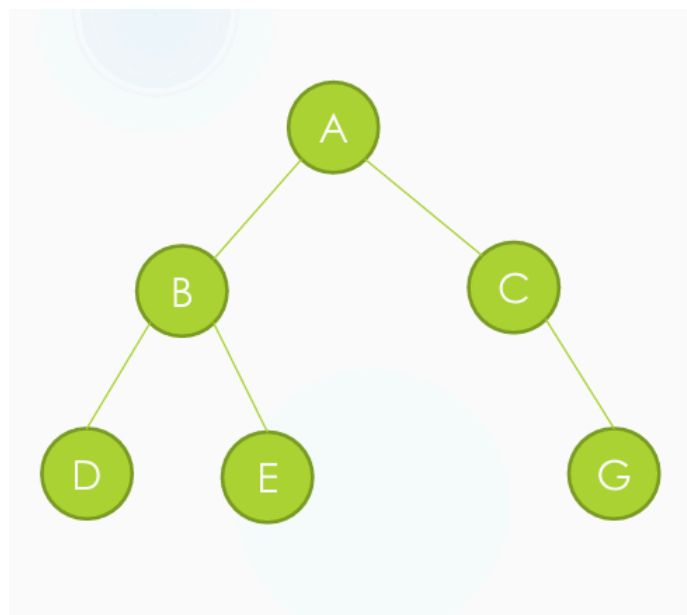
트리 탐색 및 미로찾기 수업노트

길찾기 알고리즘을 배우기 전에

길찾기 알고리즘을 배우기 전에 기초에 트리 그래프 등등을 재귀함수를 이용하여 순회 했던 방식들을 복습해보자!

기초에 배웠던 Tree 탐색 방법(2진트리 1차원 배열 기준) 다음과 같은 재귀함수를 활용해 순회가 가능하다.

트리 배열 표현





트리 구조: A-B(D,E), A-C(G)

배열 인덱스: [-, A, B, C, D, E, -, G]

```
#include <stdio.h>
char data[10] = "_ABCDE_G";
int n = 7;

void dfs(int level, int nowIndex)
{
    if (nowIndex > n || data[nowIndex] == '_')
    {
        return;
    }

    int t;
    int next;

    for (t = 0; t < 2; t++)
    {
        next = nowIndex * 2 + t;
        dfs(level + 1, next);
    }
}

int main()
{
    dfs(0, 1);
    return 0;
}
```

하지만 위 방식은 단점이 존재한다. 의도치 않은 마지막 노드에서 한꺼번에 더 진입한후 나오는 불필요한 스택면상이 사용되게 된다.

이방식을 보완해주려면 조건문을 재귀함수 호출전에 이용해서 보완이 가능하다.

DFS 복습하기

탐색하는 순서대로 노드를 출력하고 있다 (전위 순회)

```
#include <iostream>

char tree[256] = { 0, 'A', 'B', 'C', 'D', 'E', 0, 'G'};
int n = 7;

void dfs(int level, int nowIdx)
{
    int left = nowIdx * 2;
    int right = nowIdx * 2 + 1;

    std::cout << tree[nowIdx] << " ";
    if (left <= n && tree[left] != 0)
    {
        dfs(level + 1, left);
    }

    if (right <= n && tree[right] != 0)
    {
        dfs(level + 1, right);
    }
}

int main()
{
    dfs(0, 1);

    return 0;
}
```

마지막 노드만 출력하기 (중위순회)

```

#include <iostream>

char tree[256] = { 0, 'A', 'B', 'C','D','E', 0, 'G'};
int n = 7;

void dfs(int level, int nowIdx)
{
    int left = nowIdx * 2;
    int right = nowIdx * 2 + 1;

    if (left <= n && tree[left] != 0)
    {
        dfs(level + 1, left);
    }
    std::cout << tree[nowIdx] << " ";
    if (right <= n && tree[right] != 0)
    {
        dfs(level + 1, right);
    }

}

int main()
{
    dfs(0, 1);

    return 0;
}

```

마지막 노드 도착했을때 경로 출력하기(후위순회)

```

#include <iostream>

char tree[256] = { 0, 'A', 'B', 'C','D','E', 0, 'G'};
int n = 7;

```

```

void dfs(int level, int nowIdx)
{
    int left = nowIdx * 2;
    int right = nowIdx * 2 + 1;

    if (left <= n && tree[left] != 0)
    {
        dfs(level + 1, left);
    }

    if (right <= n && tree[right] != 0)
    {
        dfs(level + 1, right);
    }
    std::cout << tree[nowIdx] << " ";
}

int main()
{
    dfs(0, 1);

    return 0;
}

```

마지막 노드 도착했을때 경로 출력하기

```

char value[6] = "TBECD";
char path[6] = "";

int map[5][5] =
{
    0,1,1,0,0,
    0,0,0,1,1,
    0,0,0,0,0,
    0,0,0,0,0,
    0,0,0,0,0,
};

```

```

void run(int now, int level)
{
    std::cout << value[now];

    for (size_t i = 0; i < 5; i++)
    {
        if (map[now][i] == 1)
        {
            path[level + 1] = value[i];
            run(i, level + 1);
            path[level + 1] = 0;
        }
    }
}

int main()
{
    path[0] = 'T';
    run(0, 0);
}

```

인접행렬로 이루어진 트리 탐색하기

```

#include <iostream>
#include <map>

char data[10] = "ABCDEFGH";
int tree[6][6] = {
    {0, 1, 1, 0, 0, 0},
    {0, 0, 0, 1, 1, 0},
    {0, 0, 0, 0, 0, 1},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0},
    {0, 0, 0, 0, 0, 0}
};

```

```

//int n = 6;

void dfs(int level, int now)
{
    std::cout << data[now];

    for (int i = 0; i < 6; i++)
    {
        if (tree[now][i] == 1)
        {
            dfs(level + 1, i);
        }
    }
}

int main()
{
    dfs(0, 0);

    return 0;
}

//DFS 복습 해보기
//트리일때 (1차원 배열일때)
// 인접행렬일때
// 그래프 DFS (복습해보기)

```

미로찾기

기본 개념

시작지점에서 도착지점까지 가는 경로를 알아내는 문제

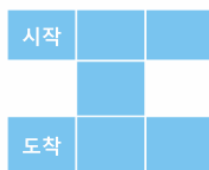
기존에 우리가 배웠던 DFS 알고리즘을 이용해서 길찾기를 해볼 생각이다.

기존에는 DFS 알고리즘, 재귀함수를 이용하여 경로수를 모두탐색하는 방식이다.

길찾기도 좌우 상하 네개의 경우의 수를 모두 DFS탐색하고 보면된다.

DFS 미로찾기 구현

```
void dfs(int level, int nowX, int nowY)
```



map



via



만약에 원하는 위치에 도착했다면 "도착"을 출력해보자.

길찾기도 그래프 탐색과 마찬가지로 이미 방문했던곳을 재귀호출해야 한다.

실제 구현 코드

```
#include <iostream>

int map[3][3] =
{
    0, 0, 0,
    1, 0, 1,
    0, 0, 0,
};

int visited[3][3] =
{
};

void findPathByDFS(int level, int x, int y)
{
    if (x == 2 && y == 2)
    {
        std::cout << "도착";
        return;
    }

    const int direct[4][2] =
    {
        {0, 1},
        {1, 0},
        {0, -1},
        {-1, 0}
    };

    for (int i = 0; i < 4; i++)
    {
        int dx = x + direct[i][0];
        int dy = y + direct[i][1];
```

```

// 맵을 넘어가는 경우
if (dx < 0 || dx >= 3 || dy < 0 || dy >= 3)
    continue;

// 벽에 부딪히는 경우, 방문한 경우
if (map[dy][dx] == 0 && visited[dy][dx] == 0)
{
    visited[dy][dx] = 1;
    findPathByDFS(level + 1, dx, dy);
    visited[dy][dx] = 0;
}
}
}

int main()
{
    visited[0][0] = 1;
    findPathByDFS(0, 0, 0);

    return 0;
}

```

순열 vs 조합

순열 (Permutation)

순열: 순서에 상관있게 선택해서 나열 하는 것

1, 2, 3 등을 뽑아서 만드는 경우의수 (순서가 중요한 것들)

조합 (Combination)

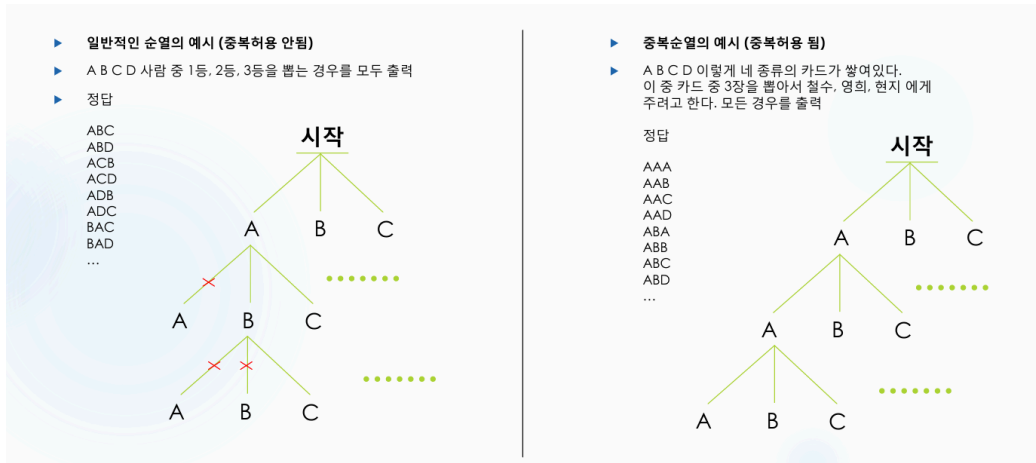
조합: 많은 것들에 여러개를 뽑아 모아서 만드는 짝

로또번호 (순서는 중요하지 않은 것)

여러명중의 참가자 선택하기

위문제들은 거의 DFS 알고리즘을 이용해서 푸는데 트리 그리는 방식이 다르다.

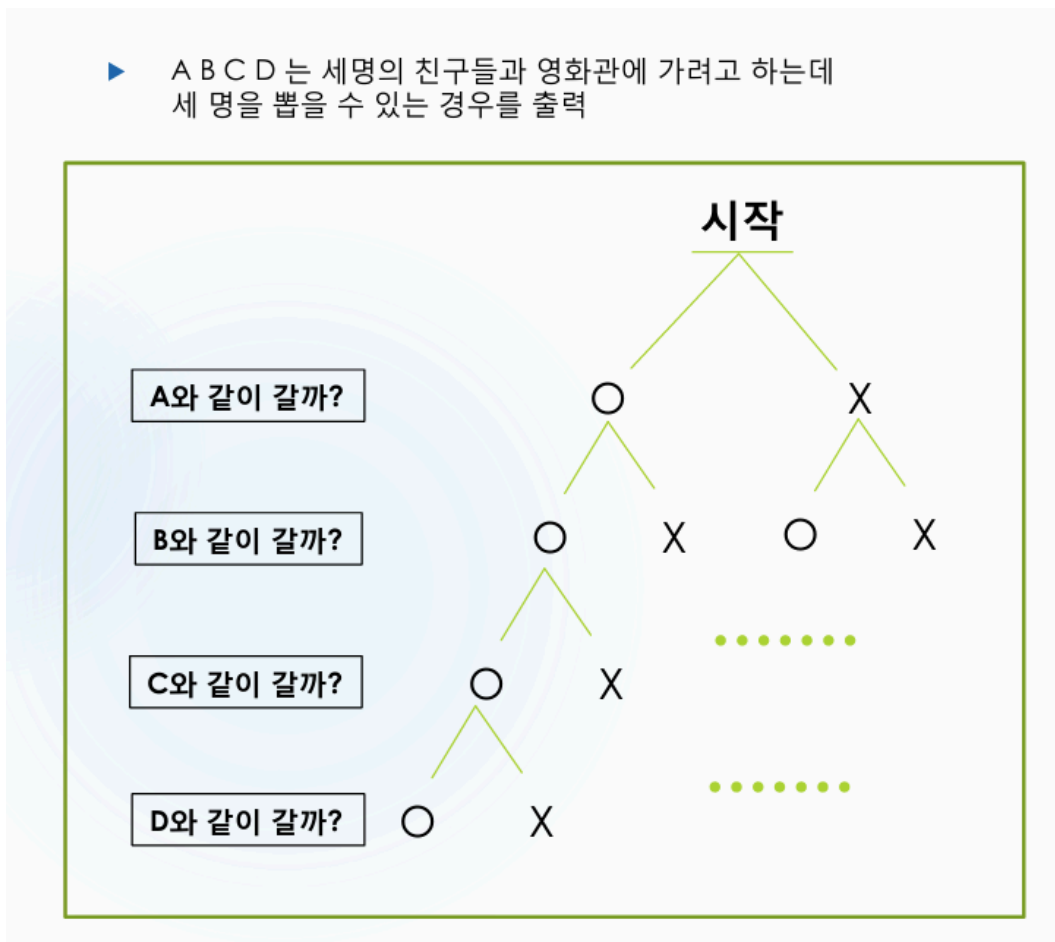
순열 예시



• 일반적인 순열의 예시 (중복허용 안함)

- A B C D 형태의 모든 순열을 출력
- 이 중 카드 중 선정 본 가지가 발견되다

조합 예시



- **중복순열의 예시 (중복허용 함)**

- A B C D 는 서로의 친구들과 영화관에 가려고 하는데
- 세 명을 뽑을 수 있는 경우의 출력

조합 구현 (DFS 활용)

```
#include <iostream>

char history[10] = {};
char ox[2] = { 'O', 'X' };
char data[10] = "ABCD";

void dfs(int level)
{
    int cnt = 0;

    if (level == 4)
    {
        //std::cout << history << std::endl;

        for (size_t i = 0; i < 4; i++)
        {
            if (history[i] == 'O')
            {
                cnt++;
            }
        }

        if (cnt == 3)
        {
            for (size_t i = 0; i < 2; i++)
            {
                if (history[i] == 'O')
                {
                    std::cout << data[i];
                }
            }
        }
    }
}
```

```

        std::cout << std::endl;
    }

    return;
}

for (size_t i = 0; i < 2; i++)
{
    history[level] = ox[i];
    dfs(level + 1);
    history[level] = 0;
}
}

int main()
{
    dfs(0);
}

```

// Brute-Force, Backtracking, DFS는 비슷한 의미로 인해 혼용에서 쓰이고 갑니다.
 // 명확한 용어의 의미는 다음과 같습니다.
 // • 완전탐색(Brute-Force): 모든 경우를 시도해보고 답을 찾아내는 방법일 뿐함. 단순 for문 돌리는 방법도 있고, 재귀호출을 쓰는 방법도 있음.
 // • Backtracking: 많은 경우를 시도하다가 정답이 될 가능성이 없으면 시도을 취소하고, 뒤로 돌아가 다른 시도을 해보는 알고리즘.

주요 개념 정리

트리 순회 방법

- 전위 순회: 루트 → 왼쪽 → 오른쪽
- 중위 순회: 왼쪽 → 루트 → 오른쪽
- 후위 순회: 왼쪽 → 오른쪽 → 루트

미로찾기 핵심

- 4방향 탐색: 상하좌우 이동

- **visited 배열:** 중복 방문 방지
- **백트래킹:** 탐색 후 원상복구
- **경계 검사:** 맵 범위 넘어가지 않도록

순열과 조합

- **순열:** 순서가 중요 (A,B,C vs B,A,C 다름)
- **조합:** 순서가 무관 (A,B,C vs B,A,C 같음)
- **구현:** 모두 DFS + 백트래킹으로 해결

알고리즘 용어

- **완전탐색(Brute-Force):** 모든 경우의 수 탐색
- **백트래킹(Backtracking):** 조건에 맞지 않으면 되돌아가기
- **DFS:** 깊이 우선 탐색으로 구현

이 모든 기법들은 **재귀함수와 백트래킹**을 기본으로 하며, 문제 상황에 따라 적절히 응용하여 사용합니다.

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 '**연습**'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얀얀코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얀얀코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

"강의는 많은데, 왜 나는 아직도 코드를 못 짤까?"

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 손이 안 움직이고,
- 복습을 하려 해도 무엇을 다시 봐야 할지 모르겠고,
- 질문할 곳도 없고,
- 유튜브는 결국 정답을 따라 치는 것밖에 안 되는 것 같고.

문제는 '연습'이 빠졌기 때문입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 얌얌코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

얌얌코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 쪼개고 설계하는 힘
- 다양한 조건을 만족시키는 실제 구현 능력
- 기능 단위가 아닌, 프로그램 단위로 사고하는 습관
- 마침내 자신의 힘으로 코드를 끝까지 작성하는 경험

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 훈련,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [얌얌코딩 상담방](#)

▼ oopy:hide

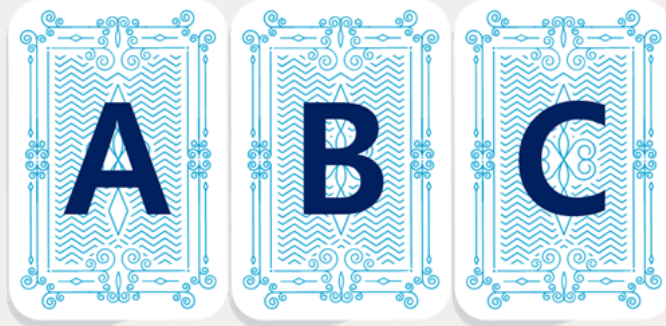
문제풀이



문제 1번

문자가 적혀 있는 세 장의 카드를 입력 받습니다.

이 세 장의 카드를 이용하여 만들수 있는 순열을 출력 해 주세요.



같은 알파벳 카드는 등장하지 않습니다.

재귀호출을 사용하여 구현 해 주세요.

입력 예제

A B C

출력 결과

ABC

ACB

BAC

BCA

CAB

CBA



문제 2번

숫자 n 을 입력 받으세요.

0~9 사이의 숫자 n 개를 사용해서 모두 더했을때
7이 나오는 경우가 총 몇가지인지 출력하세요.

[세부조건]

1. $2 \leq n \leq 6$
2. brute force로 풀어주세요.

입력 예제

3

출력 결과

36



문제 3번

남자친구와 피크닉을 위해 김밥을 만들고자 합니다.



n개의 재료를 문자열로 입력 받아주세요.

(입력받는 재료의 종류는 모두 다릅니다.)

n개의 재료 중 3개를 선택해야 합니다.

같은 재료 넣어도 될 때, 총 가능한 모든 조합(경우의수)을 모두 출력 해 주세요.

입력 예제

ABCD

출력 결과

AAA

AAB

AAC

AAD

ABB

ABC

ABD

ACC

ACD

ADD

BBB

BBC

BBD

BCC

BCD

BDD

CCC

CCD

CDD

DDD



문제 4번

동물병원 선생님은 동물들에게 밥을 주려고 합니다.

여섯 마리의 동물이 밥을 먹기 위해 줄을 섭니다.

강아지 뽀삐는 a등, 또는 b등이 되는 것을 싫어 합니다.

뽀삐가 a등 또는 b등을 할 경우를 제외하고, 동물들이 줄을 설 수 있는 경우가 총 몇개인지

Backtracking 으로 Counting 해주세요.



[입력]

뽀삐가 피하고자 하는 등수 a, b가 입력 됩니다.

[출력]

뽀삐가 a, b 등수가 되는 경우를 제외한 나머지의 경우의 수를 출력해주세요.

입력 예제

1 6

출력 결과

480



문제 5번

숫자 카드 6개를 입력 받으세요. [숫자 범위 : 1 ~ 9]
이 중 네 장을 뽑아 만들 수 있는 숫자를 만드려합니다.



만들 수 있는 네 자리 숫자의 총 개수를 출력하세요.
그리고 만들어지는 숫자 중 짝수 개수와 홀수 개수를 재귀호출을 이용하여 각각 구하세요.

[숫자 조건]

- 0은 들어갈 수 없습니다.
- 숫자 중복을 허용합니다. (ex. 1134)
- 똑같은 숫자를 세면 안됩니다.

[입력]

숫자 6 개를 입력하세요.

[출력]

만들 수 있는 숫자의 총 개수, 만들어지는 숫자들 중에서 짝수 개수와 홀수 개수를 출력하세요.

입력 예제

235761

출력 결과

1296 432 864



문제 6번

아래 단어들을 적절히 사용해서, 문장을 구성해 주세요.
사용할 수 있는 단어들은 다음과 같습니다.

BTS	SBS	BS	CBS	SES
-----	-----	----	-----	-----

위 단어들을 최소로 사용하여 입력받은 문장을 만드려고 합니다.
단어를 중복사용할 수 있습니다.

[입력]

문장을 입력 해주세요. (최대 30글자)

[출력]

문장을 만들 수 있는 단어의 사용 최소 개수를 출력하세요.

입력 예제

CBSSSES

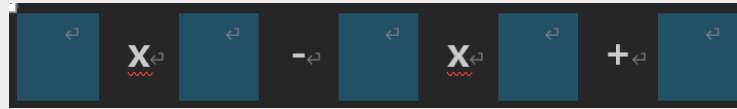
출력 결과

2



문제 7번

5개의 항에 숫자를 채우려고 합니다.



숫자 5개를 입력받고, 최고의 위치를 선정 해 주세요. ($-10 \leq \text{숫자} \leq 10$)
만들어질 수 있는 최댓값과 최솟값을 Backtracking을 이용하여 구해주세요.

(연산자 우선순위를 지켜주세요, 곱셈은 뺄셈보다 우선합니다.)

만약 1 2 3 4 5를 입력받았다면

$$\text{최댓값} = (5 \times 4) - (2 \times 1) + 3 = 21$$

$$\text{최솟값} = (3 \times 1) - (5 \times 4) + 2 = -15$$

입력 예제

1 2 3 4 5

출력 결과

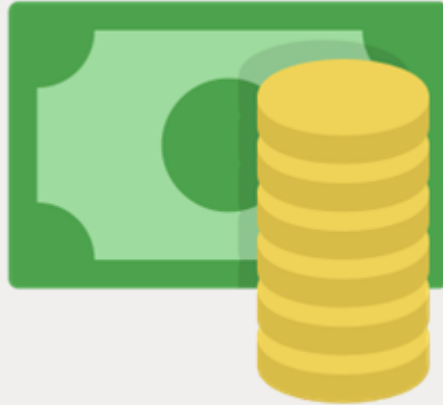
21

-15



문제 8번

세 종류의 동전이 있습니다.



교환할 지폐의 금액을 입력 받아주세요

위 동전으로 최소 몇 개의 동전을 거슬러 줄 수 있는지,

최소 수량을 구해 출력하세요.

예로들어 80원을 거슬러 주기 위해서는 40원짜리 동전 2개로 거슬러 줄 수 있습니다.

입력 예제

80

출력 결과

2



문제 9번

아래 배열은 각 글자별로 얻을 수 있는 점수들입니다.

이제 한 문자열을 입력 받습니다. 이 중에서 n 개의 글자를 선택하여 제거 해주세요.

남아있는 글자들의 점수들을 모두 구했을 때, 반드시 합이 홀수가 되어야 합니다.

n 개의 글자를 제거하고, 남은 글자들의 점수를 전부 더했을 때 최대값이 될 수 있는 값을 출력 해 주세요.

a	15
b	20
c	45
d	22
e	55
f	16
g	45

아래와 같이 입력받았고

abcae

2

만약 a, b를 제거했다면 남은 글자는 cae 이며 총 합은 $c(45) + a(15) + e(55) = 115$ 입니다.

만약 b, c를 제거했다면 남은 글자는 aae 이며 총 합은 $a(15) + a(15) + e(55) = 85$ 입니다.

만약 a, a를 제거했다면 남은 글자는 bce 이며 총 합은 $b(20) + c(45) + e(55) = 120$ 인데 홀수이기 때문에 이 선택은 무효입니다.

여기서 최대로 얻을 수 있는 점수는 115점 이므로 정답은 115 입니다.

입력 예제

abcae

2

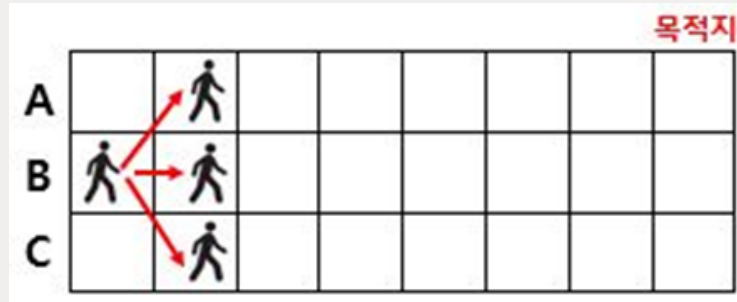
출력 결과

115



문제 10번

술주정뱅이는 목적지까지 걸어가는데 비틀비틀 움직입니다.

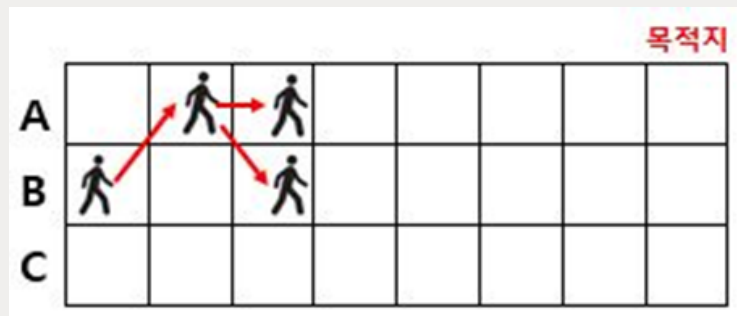


세방향으로 움직일 수 있습니다.

술주정뱅이가 만약 B에서 출발한다면

첫걸음에 갈 수 있는 곳은 (0,1), (1,1), (2,1) 입니다.

만약 (0,1)에 도착한다면 (2,0) 또는 (2,1)로 움직일 수 있습니다.



이렇게 술주정뱅이가 비틀비틀 움직일때

목적지에 도착할 수 있는 경우의 수가 총 몇 가지 존재하는지 출력하세요.

입력 예제

A

출력 결과

408